# Midterm Group Project - Part 1: Operational Databases, Data Warehousing, and ETL

## Part 1: Environment Setup

### Task 1.1: Start the Docker Compose Environment

```
[+] up 10/10
 ✔Image part01-etl-service      Built                                                                    22.4s
 ✔Network part01_week03_network Created                                                                  0.0s
 ✔Volume part01_sales_data      Created                                                                  0.0s
 ✔Volume part01_hr_data         Created                                                                  0.0s
 ✔Volume part01_warehouse_data  Created                                                                  0.0s
 ✔Container week03_sales_db     Healthy                                                                  10.8s
 ✔Container week03_hr_db        Healthy                                                                  11.3s
 ✔Container week03_warehouse_db Healthy                                                                  11.3s
 ✔Container week03_pgadmin      Created                                                                  0.1s
 ✔Container week03_etl          Created                                                                  0.1s
PS C:\Users\ksogodogo\ism6562s26-class\midterm\part01> docker compose ps
NAME                  IMAGE                 COMMAND                  SERVICE        CREATED              STATUS                        P
ORTS
week03_hr_db          postgres:16           "docker-entrypoint.s…"   hr-db          About a minute ago   Up About a minute (healthy)   0
.0.0.0:5502->5432/tcp, [::]:5502->5432/tcp
week03_pgadmin        dpage/pgadmin4:8.13   "/entrypoint.sh"         pgadmin        About a minute ago   Up 57 seconds                 0
.0.0.0:8200->80/tcp, [::]:8200->80/tcp
week03_sales_db       postgres:16           "docker-entrypoint.s…"   sales-db       About a minute ago   Up About a minute (healthy)   0
.0.0.0:5501->5432/tcp, [::]:5501->5432/tcp
week03_warehouse_db   postgres:16           "docker-entrypoint.s…"   warehouse-db   About a minute ago   Up About a minute (healthy)   0
.0.0.0:5503->5432/tcp, [::]:5503->5432/tcp
```

### Task 1.2: Connect to Each Database via psql

Connect to the sales database and list tables:

```
PS C:\Users\ksogodogo\ism6562s26-class\midterm\part01> docker compose exec sales-db psql -U student -d sales -c '\dt'
          List of relations
 Schema |   Name    | Type  |  Owner
--------+-----------+-------+---------
 public | customers | table | student
 public | orders    | table | student
 public | products  | table | student
(3 rows)
```

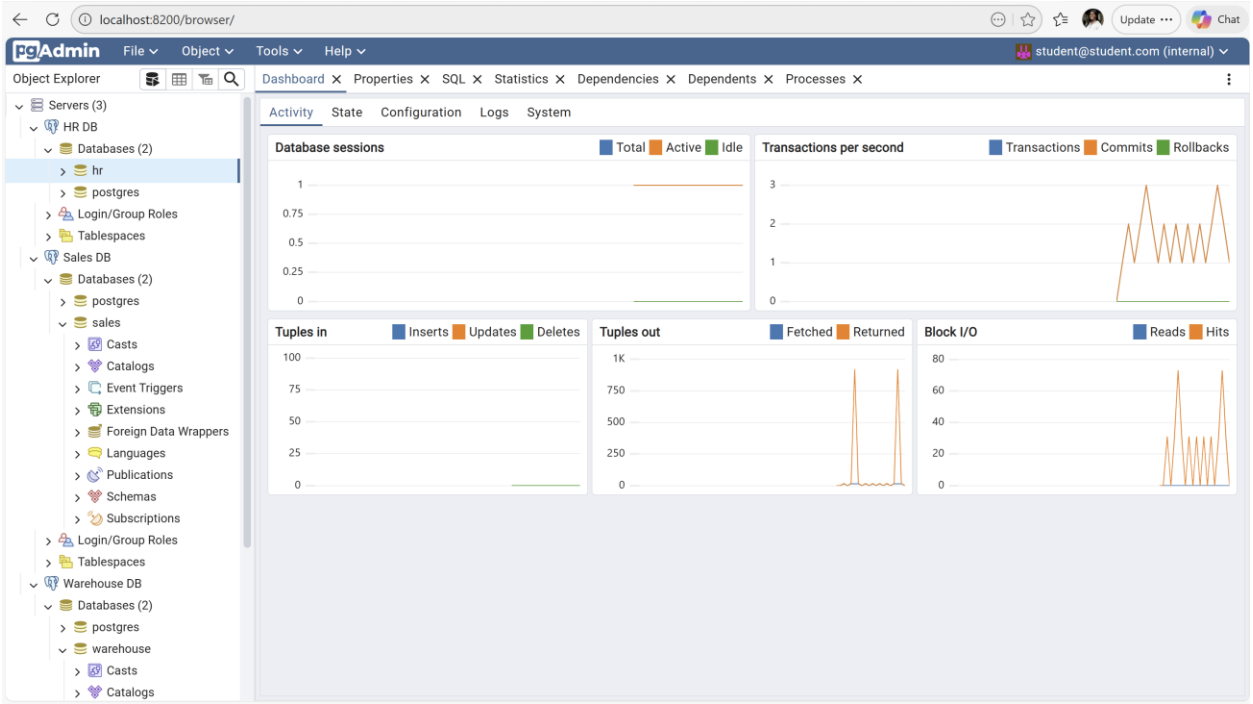
Connect to the HR database and list tables:

```
PS C:\Users\ksogodogo\ism6562s26-class\midterm\part01> docker compose exec hr-db psql -U student -d hr -c '\dt'
           List of relations
 Schema |    Name     | Type  |  Owner
--------+-------------+-------+---------
 public | departments | table | student
 public | employees   | table | student
 public | timesheets  | table | student
(3 rows)
```

Connect to the warehouse database and list tables:

```
PS C:\Users\ksogodogo\ism6562s26-class\midterm\part01> docker compose exec warehouse-db psql -U student -d warehouse -c '\dt'
            List of relations
 Schema |     Name     | Type  |  Owner
--------+--------------+-------+---------
 public | dim_customer | table | student
 public | dim_date     | table | student
 public | dim_employee | table | student
 public | dim_product  | table | student
 public | fact_sales   | table | student
(5 rows)
```

### Task 1.3: Register All Three Servers in pgAdmin




## Part 2: Explore Operational Databases

### Task 2.1: Examine the Sales Schema

### Connect to the Sales database:

```
PS C:\Users\ksogodogo\ism6562s26-class\midterm\part01> docker compose exec sales-db psql -U student -d sales
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.

sales=#
```

### Inspect sales database table structures:

Customers Table

```
sales=# \d customers
                                        Table "public.customers"
   Column    |            Type             | Collation | Nullable |                   Default
-------------+-----------------------------+-----------+----------+-----------------------------------------------
 customer_id | integer                     |           | not null | nextval('customers_customer_id_seq'::regclass)
 first_name  | character varying(100)      |           | not null |
 last_name   | character varying(100)      |           | not null |
 email       | character varying(255)      |           | not null |
 city        | character varying(100)      |           |          |
 state       | character varying(2)        |           |          |
 created_at  | timestamp without time zone |           |          | now()
Indexes:
    "customers_pkey" PRIMARY KEY, btree (customer_id)
    "customers_email_key" UNIQUE CONSTRAINT, btree (email)
    "idx_customers_email" btree (email)
    "idx_customers_state" btree (state)
Referenced by:
    TABLE "orders" CONSTRAINT "orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
```

Products Table

```
sales=# \d products
                                          Table "public.products"
    Column    |            Type             | Collation | Nullable |                   Default
--------------+-----------------------------+-----------+----------+-----------------------------------------------
 product_id   | integer                     |           | not null | nextval('products_product_id_seq'::regclass)
 product_name | character varying(200)      |           | not null |
 category     | character varying(100)      |           | not null |
 unit_price   | numeric(10,2)               |           | not null |
 created_at   | timestamp without time zone |           |          | now()
Indexes:
    "products_pkey" PRIMARY KEY, btree (product_id)
    "idx_products_category" btree (category)
Referenced by:
    TABLE "orders" CONSTRAINT "orders_product_id_fkey" FOREIGN KEY (product_id) REFERENCES products(product_id)
```

Orders Table

```
sales=# \d orders
                                  Table "public.orders"
   Column    |     Type      | Collation | Nullable |                 Default
-------------+---------------+-----------+----------+-----------------------------------------
 order_id    | integer       |           | not null | nextval('orders_order_id_seq'::regclass)
 customer_id | integer       |           | not null |
 product_id  | integer       |           | not null |
 quantity    | integer       |           | not null |
 total_price | numeric(10,2) |           | not null |
 order_date  | date          |           | not null |
Indexes:
    "orders_pkey" PRIMARY KEY, btree (order_id)
    "idx_orders_customer_id" btree (customer_id)
    "idx_orders_order_date" btree (order_date)
    "idx_orders_product_id" btree (product_id)
Check constraints:
    "orders_quantity_check" CHECK (quantity > 0)
Foreign-key constraints:
    "orders_customer_id_fkey" FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
    "orders_product_id_fkey" FOREIGN KEY (product_id) REFERENCES products(product_id)
```

In the sales database, each table, Customers, Products and Orders, has a column with a unique identifier as the primary key, which are respectively customer_id, product_id, and order_id. Additionally, the sales schema puts indexes on columns that are most likely to appear in queries, such as foreign keys, order date, email, etc. Some indexes also have constraints, like the one on email in the customers table that both speeds up searches and makes sure no two customers share the same email.

Here, the sales schema demonstrates normalization and avoids data redundancy in multiple ways. For instance, the Orders table uses foreign keys to connect to the products and customers tables. This way, customers information is only stored in the Customers table and every order only has a customer_id instead of repeating a customer's name and address on every order. Similarly, for Products, the order only has the product_id, not all details about the product. This ensures that everything is stored only once and avoids duplicate data.

### Task 2.2: Examine the HR Schema

### Connect to the HR database

```
sales=# exit
PS C:\Users\ksogodogo\ism6562s26-class\midterm\part01> docker compose exec hr-db psql -U student -d hr
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.
```

**Inspect HR database table structures:**

Departments Table

```
hr=# \d departments
                                          Table "public.departments"
     Column      |          Type          | Collation | Nullable |                     Default
-----------------+------------------------+-----------+----------+-----------------------------------------------------
 department_id   | integer                |           | not null | nextval('departments_department_id_seq'::regclass)
 department_name | character varying(100) |           | not null |
 location        | character varying(100) |           | not null |
Indexes:
    "departments_pkey" PRIMARY KEY, btree (department_id)
    "departments_department_name_key" UNIQUE CONSTRAINT, btree (department_name)
Referenced by:
    TABLE "employees" CONSTRAINT "employees_department_id_fkey" FOREIGN KEY (department_id) REFERENCES departments(department_id)
```

Employee Table

```
hr=# \d employees
                                          Table "public.employees"
    Column     |          Type          | Collation | Nullable |                     Default
---------------+------------------------+-----------+----------+-----------------------------------------------
 employee_id   | integer                |           | not null | nextval('employees_employee_id_seq'::regclass)
 first_name    | character varying(100) |           | not null |
 last_name     | character varying(100) |           | not null |
 email         | character varying(255) |           | not null |
 department_id | integer                |           | not null |
 hire_date     | date                   |           | not null |
 job_title     | character varying(100) |           | not null |
Indexes:
    "employees_pkey" PRIMARY KEY, btree (employee_id)
    "employees_email_key" UNIQUE CONSTRAINT, btree (email)
    "idx_employees_department_id" btree (department_id)
    "idx_employees_hire_date" btree (hire_date)
Foreign-key constraints:
    "employees_department_id_fkey" FOREIGN KEY (department_id) REFERENCES departments(department_id)
Referenced by:
    TABLE "timesheets" CONSTRAINT "timesheets_employee_id_fkey" FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
```

Timesheet Table

```
hr=# \d timesheets
                                        Table "public.timesheets"
    Column     |     Type     | Collation | Nullable |                     Default
---------------+--------------+-----------+----------+-----------------------------------------------
 timesheet_id | integer      |           | not null | nextval('timesheets_timesheet_id_seq'::regclass)
 employee_id  | integer      |           | not null |
 work_date    | date         |           | not null |
 hours_worked | numeric(4,2) |           | not null |
Indexes:
    "timesheets_pkey" PRIMARY KEY, btree (timesheet_id)
    "idx_timesheets_employee_id" btree (employee_id)
    "idx_timesheets_work_date" btree (work_date)
    "timesheets_employee_id_work_date_key" UNIQUE CONSTRAINT, btree (employee_id, work_date)
Check constraints:
    "timesheets_hours_worked_check" CHECK (hours_worked >= 0::numeric AND hours_worked <= 24::numeric)
Foreign-key constraints:
    "timesheets_employee_id_fkey" FOREIGN KEY (employee_id) REFERENCES employees(employee_id)
```

The HR database also has three tables: Departments, Employees and Timesheets; each have a primary key, which are department_id, employee_id, and timesheet_id, respectively. Here, some of these indexes prevent duplicate department entries on email and name, and there is the UNIQUE(employee_id, work_date) constraint on timesheets that stops the same employee from recording hours multiple times on the same day. This helps keep the data accurate, consistent and reduce mistakes to ensure the information can be trusted. With all the rules enforced, similar to the sales database, everything is central; department details are in Departments table, employee details are in Employee table and Employees use IDs to refer back to the department table and Timesheets uses IDs to refer back to employees table.

## Task 2.3: Run Queries Against the Operational Database

### Sales Database Queries:

-- Query 1: Total revenue by product category

```
sales=# SELECT p.category,
       COUNT(*) AS order_count,
       SUM(o.total_price) AS total_revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY total_revenue DESC;
  category   | order_count | total_revenue
-------------+-------------+---------------
 Electronics |          15 |       7199.81
 Furniture   |           7 |       2789.92
 Books       |           5 |        395.91
(3 rows)
```

-- Query 2: Customers with the most orders

```
sales=# SELECT c.first_name, c.last_name, c.state,
       COUNT(*) AS order_count
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.state
ORDER BY order_count DESC;
 first_name | last_name | state | order_count
------------+-----------+-------+-------------
 Alice      | Johnson   | FL    |           5
 Carol      | Williams  | GA    |           4
 Bob        | Smith     | FL    |           4
 Eva        | Davis     | NC    |           3
 David      | Brown     | FL    |           3
 Frank      | Miller    | FL    |           3
 Grace      | Wilson    | TN    |           3
 Henry      | Taylor    | GA    |           2
(8 rows)
```

### HR Database Queries:

-- Query 3: Employee count per department

```
hr=# SELECT d.department_name, d.location,
       COUNT(*) AS employee_count
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY d.department_id, d.department_name, d.location
ORDER BY employee_count DESC;
 department_name | location | employee_count
-----------------+----------+----------------
 Human Resources | Tampa    |              3
 Marketing       | Orlando  |              3
 Sales           | Miami    |              3
 Engineering     | Tampa    |              3
(4 rows)
```

-- Query 4: Total hours worked per department (Jan 2026)

```
hr=# SELECT d.department_name,
       SUM(t.hours_worked) AS total_hours,
       ROUND(AVG(t.hours_worked), 1) AS avg_hours_per_entry
FROM timesheets t
JOIN employees e ON t.employee_id = e.employee_id
JOIN departments d ON e.department_id = d.department_id
WHERE t.work_date BETWEEN '2026-01-01' AND '2026-01-31'
GROUP BY d.department_id, d.department_name
ORDER BY total_hours DESC;
 department_name | total_hours | avg_hours_per_entry
-----------------+-------------+---------------------
 Engineering     |      158.00 |                 7.9
 Sales           |      139.50 |                 7.8
 Marketing       |      133.00 |                 7.8
 Human Resources |      131.00 |                 7.7
(4 rows)
```

# Part 3: Data Warehouse Concepts

### Task 3.1: Examine the Star Schema

Connect to the warehouse database and inspect all table structures:

\d dim_date

```
warehouse=# \d dim_date
                    Table "public.dim_date"
    Column     |         Type          | Collation | Nullable | Default
---------------+-----------------------+-----------+----------+---------
 date_key      | integer               |           | not null |
 full_date     | date                  |           | not null |
 year          | integer               |           | not null |
 quarter       | integer               |           | not null |
 month         | integer               |           | not null |
 month_name    | character varying(20) |           | not null |
 day_of_month  | integer               |           | not null |
 day_of_week   | integer               |           | not null |
 day_name      | character varying(20) |           | not null |
 is_weekend    | boolean               |           | not null |
Indexes:
    "dim_date_pkey" PRIMARY KEY, btree (date_key)
    "dim_date_full_date_key" UNIQUE CONSTRAINT, btree (full_date)
Referenced by:
    TABLE "fact_sales" CONSTRAINT "fact_sales_date_key_fkey" FOREIGN KEY (date_key) REFERENCES dim_date(date_key)
```

```
    "dim_customer_source_customer_id_key" UNIQUE CONSTRAINT, btree (source_customer_id)
Referenced by:
    TABLE "fact_sales" CONSTRAINT "fact_sales_customer_key_fkey" FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key)
```

\d dim_product

```
warehouse=# \d dim_product
                                        Table "public.dim_product"
      Column       |          Type          | Collation | Nullable |                    Default
-------------------+------------------------+-----------+----------+-----------------------------------------------
 product_key       | integer                |           | not null | nextval('dim_product_product_key_seq'::regclass)
 source_product_id | integer                |           | not null |
 product_name      | character varying(200) |           | not null |
 category          | character varying(100) |           | not null |
 unit_price        | numeric(10,2)          |           | not null |
Indexes:
    "dim_product_pkey" PRIMARY KEY, btree (product_key)
    "dim_product_source_product_id_key" UNIQUE CONSTRAINT, btree (source_product_id)
Referenced by:
    TABLE "fact_sales" CONSTRAINT "fact_sales_product_key_fkey" FOREIGN KEY (product_key) REFERENCES dim_product(product_key)
```

\d dim_employee

```
warehouse=# \d dim_employee
                                         Table "public.dim_employee"
       Column        |          Type          | Collation | Nullable |                     Default
---------------------+------------------------+-----------+----------+-----------------------------------------------
 employee_key        | integer                |           | not null | nextval('dim_employee_employee_key_seq'::regclass)
 source_employee_id  | integer                |           | not null |
 first_name          | character varying(100) |           | not null |
 last_name           | character varying(100) |           | not null |
 email               | character varying(255) |           | not null |
 job_title           | character varying(100) |           | not null |
 department_name     | character varying(100) |           | not null |
 department_location | character varying(100) |           | not null |
Indexes:
    "dim_employee_pkey" PRIMARY KEY, btree (employee_key)
    "dim_employee_source_employee_id_key" UNIQUE CONSTRAINT, btree (source_employee_id)
```

\d fact_sales

```
warehouse=# \d fact_sales
                                Table "public.fact_sales"
      Column       |     Type      | Collation | Nullable |                   Default
-------------------+---------------+-----------+----------+-----------------------------------------
 sale_key          | integer       |           | not null | nextval('fact_sales_sale_key_seq'::regclass)
 date_key          | integer       |           | not null |
 customer_key      | integer       |           | not null |
 product_key       | integer       |           | not null |
 source_order_id   | integer       |           | not null |
 quantity          | integer       |           | not null |
 unit_price        | numeric(10,2) |           | not null |
 total_price       | numeric(10,2) |           | not null |
Indexes:
    "fact_sales_pkey" PRIMARY KEY, btree (sale_key)
    "fact_sales_source_order_id_key" UNIQUE CONSTRAINT, btree (source_order_id)
    "idx_fact_sales_customer_key" btree (customer_key)
    "idx_fact_sales_date_key" btree (date_key)
    "idx_fact_sales_product_key" btree (product_key)
Foreign-key constraints:
    "fact_sales_customer_key_fkey" FOREIGN KEY (customer_key) REFERENCES dim_customer(customer_key)
    "fact_sales_date_key_fkey" FOREIGN KEY (date_key) REFERENCES dim_date(date_key)
    "fact_sales_product_key_fkey" FOREIGN KEY (product_key) REFERENCES dim_product(product_key)
```

**Task 3.2: Compare Operational vs Warehouse Design**

Compared to the HR and sales databases, where data is split across multiple tables with no duplication, the data warehouse brings everything together in one place in a denormalized format. In the sales database, customers and products are stored as separate normalized tables, and write operations happen there directly. On the other hand, the warehouse database receives copies of data from the sales database and puts them into *dim_customer* and *dim_product* tables and assign them new keys. The old key are preserved as *source_customer_id* and *source_product_id* for traceability, but they are not the keys the warehouse uses. Additionally data from the HR and sales databases is loaded into the data warehouse in bulk periodically.

In the warehouse database, you can directly find which department an employee belongs to because the department name and location are stored together in the *dim_employee* table. However, in the HR database, you have to join the employees and departments tables every time you need to get this information. This is where denormalization comes into play. It allows the warehouse to store everything in one place so queries do not have to look at multiple tables and this makes analysis much faster. Moreover, if an employee changes departments, the data warehouse keeps history by maintaining records of where they were before.

The warehouse has a *dim_date* table instead of simply having an *order_date* column because a date by itself is not detailed enough to answer all the queries we might have. For instance, without this separation, you would need to calculate and do extra steps every time you want specific days, months, or to check whether a day was a weekend. *dim_date* simplifies all of that and allows the queries to just look them up instantly, which makes a big difference, especially when you have large amounts of data.

**Task 3.3: Identify Facts vs Dimensions**

As mentioned in the notes, fact tables store measurable business events with numeric measures, while dimension tables store the descriptive context around those facts. In data warehouse, the fact_sales table is the fact table, and dim_customer, dim_product, dim_date, and dim_employee are the dimension tables. They are easy to distinguish because *fact_sales* contains numbers that can be measured and combined, like quantity, unit_price, and total_price. The dimension tables contain descriptive attributes such as names, categories, and locations. The fact table also has all the foreign keys that link to the dimension tables.

Each row in fact_sales table represents one customer buying one product on a specific date and that is the grain of the fact_sales table. A business question that an analyst could answer by joining fact_sales with some dimension tables is 'which product category

generated the most sales in the first week of February'. To answer that, we would join fact_sales with dim_product and dim_date. fact_sales will give us the total_price to sum up, dim_product will give us the category to group by and dim_date will let us filter by the specific dates in that first week. In summary, we would need three tables: fact_sales, dim_product and dim_date.

## Part 4: ETL Pipeline

### Task 4.1: Analyze the ETL Script

```
Container week03_hr_db Waiting
Container week03_warehouse_db Waiting
Container week03_hr_db Healthy
Container week03_sales_db Healthy
Container week03_warehouse_db Healthy
Container part01-etl-service-run-c9a78ef0d52d Creating
Container part01-etl-service-run-c9a78ef0d52d Created
============================================================
ETL Pipeline Starting
============================================================

[EXTRACT] Reading from operational databases...
  Extracted 8 customers from sales-db
  Extracted 10 products from sales-db
  Extracted 27 orders from sales-db
  Extracted 12 employees from hr-db

[TRANSFORM] Preparing dimension data...
  Generated 365 dim_date rows for 2026
  Employee dimension: denormalized with department info (done in extract query)

[LOAD] Writing to data warehouse...
  Loaded dim_date (365 rows, conflicts skipped)
  Loaded dim_customer (8 rows, upserted)
  Loaded dim_product (10 rows, upserted)
  Loaded dim_employee (12 rows, upserted)
  Loaded fact_sales (27 inserted, 0 already existed)

============================================================
ETL Pipeline Complete
============================================================
```

```
PS C:\Users\ksogodogo\ism6562s26-class\midterm\part01> docker compose run etl-service cat /etc/cron.d/etl-cron
time="2026-03-29T20:36:00-04:00" level=warning msg="Found orphan containers ([part01-etl-service-run-c9a78ef0d52d part01-etl-service-run
-3f0a11ce169a]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remov
e-orphans flag to clean it up."
[+]  3/3t 3/33
 ✔Container week03_warehouse_db Running                                                                                    0.0s
 ✔Container week03_hr_db        Running                                                                                    0.0s
 ✔Container week03_sales_db     Running                                                                                    0.0s
Container week03_hr_db Waiting
Container week03_warehouse_db Waiting
Container week03_sales_db Waiting
Container week03_hr_db Healthy
Container week03_sales_db Healthy
Container week03_warehouse_db Healthy
Container part01-etl-service-run-45f4816c6261 Creating
Container part01-etl-service-run-45f4816c6261 Created
# Run the ETL pipeline every 5 minutes, sending output to Docker logs
*/5 * * * * cd /app && /usr/local/bin/python etl_script.py >> /proc/1/fd/1 2>&1
```

The ETL pipeline shows three main processes. First, we have the Extract phase where data is pulled from both operational databases: from sales-db, we have customers, products, and orders and from hr-db, we have employees and departments together using a join. The script uses standard SELECT and SELECT + JOIN queries to pull all rows from each table. The output shows: 8 customers, 10 products, 27 orders and 12 employees extracted.

Next, in the Transform phase, there are three transformations happening. First, for *dim_date*, instead of pulling dates from the source database, the script has pre-calculated information for every day of 2026 to generate the quarter, month, day, and whether a day is a weekend. Then, customers and products are copied into *dim_customer* and *dim_product* with new

IDs. Lastly, employees and departments are denormalized into *dim_employee*, with department information added to each employee.

In the Load phase, ON CONFLICT clauses are used to ensure the script can run multiple times without creating duplicates because every time the ETL script runs, it tries to insert data into the warehouse. Without ON CONFLICT clauses, running it twice would create duplicate rows. They prevent this by detecting when a record already exists and skipping it instead of inserting it again. Which is what makes the script idempotent. The cron schedule */5 * * * * means the script runs every 5 minutes via cron. The */5 in the minutes position means every *5th minute*, and the remaining stars mean *every hour, every day, every month, every weekday*.

**Task 4.2: Verify ETL Results**

**Source database counts:**

```
sales=# -- Run on sales-db
SELECT 'customers' AS source_table, COUNT(*) FROM customers
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'orders', COUNT(*) FROM orders;
 source_table | count
--------------+-------
 customers    |     8
 products     |    10
 orders       |    27
(3 rows)
```

**Warehouse counts:**

```
warehouse=# -- Run on warehouse-db
SELECT 'dim_customer' AS warehouse_table, COUNT(*) FROM dim_customer
UNION ALL SELECT 'dim_product', COUNT(*) FROM dim_product
UNION ALL SELECT 'dim_employee', COUNT(*) FROM dim_employee
UNION ALL SELECT 'dim_date', COUNT(*) FROM dim_date
UNION ALL SELECT 'fact_sales', COUNT(*) FROM fact_sales;
 warehouse_table | count
-----------------+-------
 dim_customer    |     8
 dim_product     |    10
 dim_employee    |    12
 dim_date        |   365
 fact_sales      |    27
(5 rows)
```

**Task 4.3: Test the ETL Pipeline**

ETL pipeline: New data from the operational databases.

**Steps:**

Insert a new order into the sales database:

```
sales=# INSERT INTO orders (customer_id, product_id, quantity, total_price, order_date)
VALUES (2, 1, 1, 1299.99, '2026-02-25');
INSERT 0 1
```

Verify the new order exists in the sales database:

```
sales=# SELECT * FROM orders ORDER BY order_id DESC LIMIT 3;
 order_id | customer_id | product_id | quantity | total_price | order_date
----------+-------------+------------+----------+-------------+------------
       28 |           2 |          1 |        1 |     1299.99 | 2026-02-25
       27 |           7 |         10 |        1 |      349.99 | 2026-02-22
       26 |           3 |          9 |        1 |      199.99 | 2026-02-20
(3 rows)
```

Manually trigger the ETL pipeline:

```
[+]  3/3t 3/33
 ✔Container week03_warehouse_db Running                                                                    0.0s
 ✔Container week03_hr_db        Running                                                                    0.0s
 ✔Container week03_sales_db     Running                                                                    0.0s
Container week03_warehouse_db Waiting
Container week03_sales_db Waiting
Container week03_hr_db Waiting
Container week03_sales_db Healthy
Container week03_warehouse_db Healthy
Container week03_hr_db Healthy
Container part01-etl-service-run-f78bcb2a64df Creating
Container part01-etl-service-run-f78bcb2a64df Created
============================================================
ETL Pipeline Starting
============================================================

[EXTRACT] Reading from operational databases...
  Extracted 8 customers from sales-db
  Extracted 10 products from sales-db
  Extracted 28 orders from sales-db
  Extracted 12 employees from hr-db

[TRANSFORM] Preparing dimension data...
  Generated 365 dim_date rows for 2026
  Employee dimension: denormalized with department info (done in extract query)

[LOAD] Writing to data warehouse...
  Loaded dim_date (365 rows, conflicts skipped)
  Loaded dim_customer (8 rows, upserted)
  Loaded dim_product (10 rows, upserted)
  Loaded dim_employee (12 rows, upserted)
  Loaded fact_sales (0 inserted, 28 already existed)

============================================================
ETL Pipeline Complete
============================================================
```

Verify the new order appears in the warehouse:

```
warehouse=# SELECT f.sale_key, d.full_date, c.first_name, c.last_name,
       p.product_name, f.quantity, f.total_price
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
ORDER BY f.sale_key DESC
LIMIT 3;
 sale_key | full_date  | first_name | last_name |     product_name     | quantity | total_price
----------+------------+------------+-----------+----------------------+----------+-------------
       55 | 2026-02-25 | Bob        | Smith     | Laptop Pro 15        |        1 |     1299.99
       27 | 2026-02-22 | Grace      | Wilson    | Monitor 27-inch      |        1 |      349.99
       26 | 2026-02-20 | Carol      | Williams  | Noise-Cancel Headset |        1 |      199.99
(3 rows)
```

## Part 5: Analytical Queries

### Task 5.1: Write Analytical Queries

## Query 1: Monthly Sales Summary

```
SELECT
    d.month_name,
    d.year,
    COUNT(f.sale_key)        AS total_orders,
    SUM(f.total_price)       AS total_revenue
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY d.year, d.month, d.month_name
ORDER BY d.year, d.month;
```

```
warehouse=# SELECT
    d.month_name,
    d.year,
    COUNT(f.sale_key)        AS total_orders,
    SUM(f.total_price)       AS total_revenue
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
GROUP BY d.year, d.month, d.month_name
ORDER BY d.year, d.month;
 month_name | year | total_orders | total_revenue
------------+------+--------------+---------------
 January    | 2026 |           15 |       5773.78
 February   | 2026 |           13 |       5911.85
(2 rows)
```

## Query 2: Top Customers by Spending

```
SELECT
    c.first_name,
    c.last_name,
    COUNT(f.sale_key)        AS number_of_orders,
    SUM(f.total_price)       AS total_spent
FROM fact_sales f
JOIN dim_customer c ON f.customer_key = c.customer_key
GROUP BY c.customer_key, c.first_name, c.last_name
ORDER BY total_spent DESC;
```

```
warehouse=# SELECT
    c.first_name,
    c.last_name,
    COUNT(f.sale_key)        AS number_of_orders,
    SUM(f.total_price)       AS total_spent
FROM fact_sales f
JOIN dim_customer c ON f.customer_key = c.customer_key
GROUP BY c.customer_key, c.first_name, c.last_name
ORDER BY total_spent DESC;
 first_name | last_name | number_of_orders | total_spent
------------+-----------+------------------+-------------
 Bob        | Smith     |                5 |     2391.94
 Alice      | Johnson   |                5 |     2209.94
 Eva        | Davis     |                3 |     1979.96
 Frank      | Miller    |                3 |     1949.97
 Carol      | Williams  |                4 |     1737.93
 David      | Brown     |                3 |      739.95
 Grace      | Wilson    |                3 |      549.96
 Henry      | Taylor    |                2 |      125.98
(8 rows)
```

### Query 3: Revenue by Product Category and Month

```
SELECT
    p.category,
    d.month_name,
    d.year,
    SUM(f.total_price)       AS total_revenue
FROM fact_sales f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_date d    ON f.date_key    = d.date_key
GROUP BY p.category, d.month, d.month_name, d.year,
ORDER BY p.category, d.year, d.month;
```

```
warehouse=# SELECT
    p.category,
    d.month_name,
    d.year,
    SUM(f.total_price)       AS total_revenue
FROM fact_sales f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_date d    ON f.date_key    = d.date_key
GROUP BY p.category, d.year, d.month, d.month_name
ORDER BY p.category, d.year, d.month;
  category   | month_name | year | total_revenue
-------------+------------+------+---------------
 Books       | January    | 2026 |        263.94
 Books       | February   | 2026 |        131.97
 Electronics | January    | 2026 |       3699.89
 Electronics | February   | 2026 |       4799.91
 Furniture   | January    | 2026 |       1809.95
 Furniture   | February   | 2026 |        979.97
(6 rows)
```

### Task 5.2: EXPLAIN ANALYZE Comparison

### On the sales (operational) database:

```
sales=# EXPLAIN ANALYZE
SELECT p.category,
       COUNT(*) AS order_count,
       SUM(o.total_price) AS total_revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY total_revenue DESC;
                                                        QUERY PLAN
-------------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=53.21..53.48 rows=110 width=258) (actual time=0.090..0.092 rows=3 loops=1)
   Sort Key: (sum(o.total_price)) DESC
   Sort Method: quicksort  Memory: 25kB
   ->  HashAggregate  (cost=48.10..49.48 rows=110 width=258) (actual time=0.045..0.047 rows=3 loops=1)
         Group Key: p.category
         Batches: 1  Memory Usage: 24kB
         ->  Hash Join  (cost=12.47..38.58 rows=1270 width=234) (actual time=0.027..0.032 rows=28 loops=1)
               Hash Cond: (o.product_id = p.product_id)
               ->  Seq Scan on orders o  (cost=0.00..22.70 rows=1270 width=20) (actual time=0.005..0.006 rows=28 loops=1)
               ->  Hash  (cost=11.10..11.10 rows=110 width=222) (actual time=0.009..0.010 rows=10 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     ->  Seq Scan on products p  (cost=0.00..11.10 rows=110 width=222) (actual time=0.004..0.005 rows=10 loops=1)
 Planning Time: 0.787 ms
 Execution Time: 0.178 ms
(14 rows)
```

**On the warehouse database:**

```
warehouse=# EXPLAIN ANALYZE
SELECT p.category,
       COUNT(*) AS order_count,
       SUM(f.total_price) AS total_revenue
FROM fact_sales f
JOIN dim_product p ON f.product_key = p.product_key
GROUP BY p.category
ORDER BY total_revenue DESC;
                                                          QUERY PLAN
-----------------------------------------------------------------------------------------------------------------------------------
 Sort  (cost=47.16..47.43 rows=110 width=258) (actual time=0.058..0.059 rows=3 loops=1)
   Sort Key: (sum(f.total_price)) DESC
   Sort Method: quicksort  Memory: 25kB
   ->  HashAggregate  (cost=42.05..43.43 rows=110 width=258) (actual time=0.051..0.053 rows=3 loops=1)
         Group Key: p.category
         Batches: 1  Memory Usage: 24kB
         ->  Hash Join  (cost=12.47..34.78 rows=970 width=234) (actual time=0.032..0.037 rows=28 loops=1)
               Hash Cond: (f.product_key = p.product_key)
               ->  Seq Scan on fact_sales f  (cost=0.00..19.70 rows=970 width=20) (actual time=0.014..0.015 rows=28 loops=1)
               ->  Hash  (cost=11.10..11.10 rows=110 width=222) (actual time=0.011..0.011 rows=10 loops=1)
                     Buckets: 1024  Batches: 1  Memory Usage: 9kB
                     ->  Seq Scan on dim_product p  (cost=0.00..11.10 rows=110 width=222) (actual time=0.006..0.007 rows=10 loops=1)
 Planning Time: 0.215 ms
 Execution Time: 0.096 ms
(14 rows)
```

**Execution Plan Comparison**

Both queries used sequential scans across their respective tables and exactly one JOIN each: orders to products on product_id in sales-db and fact_sales to dim_product on product_key in the warehouse database. Each query had a single Hash Join, a HashAggregate grouped by category and a quicksort to order results by revenue, producing both 3 output rows. Sales-db had a startup cost of 53.21 and total cost of 53.48, while the warehouse came in lower at 47.16 and 47.43 respectively. Execution time was 0.178ms for sales-db, 0.096ms for the warehouse, and planning time was 0.787ms and 0.215ms respectively. The warehouse queries were cheaper and more efficient. Both read the 28 rows from their fact tables and 10 rows from their product tables, so the row counts were equal but the warehouse consistently did it with less overhead.

With millions of rows, the differences would be even more significant. Sequential scans would become extremely slow, and planning time differences would favor the data warehouse. This is because the warehouse is built for analytical queries that read large datasets, and its star schema reduces the number of joins needed which makes complex analytical queries faster and easier to run at scale. In contrast, the sales database requires JOINs to normalized tables whenever extra details are needed. As the tables grow larger, this results in more time spent planning and executing queries.